

Identity Search And Document Verification System

Client Presentation Notes And Technical Explanation

1. Opening Pitch

This project is an Identity Search and Digital Document Verification System. It allows an operator to search employee identity records manually, upload Aadhaar or PAN card images, extract identity information using OCR and computer vision, verify the extracted identity against a PostgreSQL database, compare the face extracted from the uploaded document with the stored database photo, and finally display a verification decision.

In simple words: the system checks whether the person shown on a government ID document exists in our internal employee database and whether the face on the uploaded document matches the stored face image for that employee.

The project is built as a modular Python application with a FastAPI backend, a Streamlit frontend, PostgreSQL for identity records, YOLO and PaddleOCR for document field extraction, and OpenCV for lightweight face verification.

2. Business Problem

Organizations often receive identity documents during onboarding, verification, audits, or internal employee checks. Manual checking is slow and error-prone. The operator has to read the document, search the database, compare values, open stored employee photos, and decide whether the identity is genuine.

This system automates that workflow:

- The operator uploads an Aadhaar or PAN image.
- OCR extracts document fields such as name, DOB, Aadhaar number, or PAN number.
- The backend verifies those values against the database.
- The system extracts the face from the uploaded ID.
- The extracted face is compared with the stored employee photo.
- A final status is displayed: VERIFIED, MANUAL REVIEW, or NOT VERIFIED.

This saves time, reduces manual mistakes, and gives a repeatable verification process.

3. What The System Can Do

Current features:

- Manual identity search by full name, DOB, Aadhaar, PAN, phone, email, employee ID, department, and state.
- Aadhaar image upload.
- PAN image upload.
- OCR-based information extraction using the existing Indian ID validator model pipeline.
- Database verification against PostgreSQL table `demodataset`.

- Aadhaar matching even when OCR returns digits without spaces and the database stores spaces.
- PAN matching with normalized uppercase text.
- Face extraction from uploaded document.
- Face comparison with database photo.
- Display of extracted document face and stored database face.
- Display of verified database record when a match is found.
- Safe error handling through API responses.

4. High-Level Architecture

The project has three main layers:

1. Frontend: Streamlit dashboard.
2. Backend: FastAPI service.
3. Data and AI layer: PostgreSQL database, local employee photos, OCR model, face comparison.

Flow:

User uploads document in Streamlit -> Streamlit sends file to FastAPI -> FastAPI saves file -> OCR model extracts fields -> database service searches PostgreSQL -> OCR model face output is compared with database photo -> decision service returns final result -> Streamlit displays result.

5. Folder Structure

Project root:

```
identity-search-service/
  requirement..txt
  backend/
    app.py
    services/
      database_service.py
      file_service.py
      ocr_service.py
      face_service.py
      decision_service.py
  frontend/
    dashboard.py
    matched_employee_photos/
      IA202610001/
        IA202610001_face.jpg
    ...
  docs/
    Identity_Search_Service_Client_Explanation.md
    Identity_Search_Service_Client_Explanation.pdf
    generate_project_pdf.py
```

External model dependency:

This folder contains:

- `inference.py`
- YOLO model files
- PaddleOCR integration
- `results/face.jpg`
- `detected\_text.json` style model output

6. Technology Stack

Main project:

- Python 3.10 environment
- FastAPI 0.136.3
- Uvicorn 0.46.0
- Streamlit 1.57.0
- psycpg2-binary 2.9.12
- python-multipart 0.0.29
- requests 2.32.3
- opencv-python 4.13.0.92
- numpy 2.2.2

External OCR/model project:

- YOLO through Ultralytics
- PaddleOCR
- PaddlePaddle
- OpenCV
- Hugging Face Hub model structure

Database:

- PostgreSQL
- Database name: `excel\_import`
- Main table: `demodataset`

7. Why Each Library Is Used

FastAPI:

Used to create backend APIs. It is fast, simple, and supports file uploads, form data, and JSON responses.

Uvicorn:

Runs the FastAPI app as an ASGI server. Command used:

```
uvicorn app:app --reload
```

Streamlit:

Used to build the dashboard quickly in Python. It provides file upload buttons, tabs, images, columns, and result display without writing HTML or JavaScript.

psycopg2-binary:

Used to connect Python with PostgreSQL and execute SQL queries.

python-multipart:

Required by FastAPI to receive uploaded files and form data.

requests:

Used by Streamlit to call the FastAPI backend endpoints.

OpenCV:

Used for image loading, face detection, face cropping, histogram comparison, resizing, and structural comparison.

NumPy:

Used with OpenCV for image arrays and mathematical score calculation.

YOLO:

Used in the external Indian ID validator project to detect fields on Aadhaar and PAN cards.

PaddleOCR:

Used in the external model pipeline to extract text from detected document regions.

8. Backend Entry Point: backend/app.py

This file is the main FastAPI application.

Important imports:

```
from fastapi import FastAPI
from fastapi import Form
from fastapi import UploadFile
from fastapi import File
from fastapi.responses import JSONResponse
```

Meaning:

- `FastAPI` creates the backend application object.
- `Form` reads form fields such as `field`, `value`, and `document\_type`.
- `UploadFile` represents an uploaded document file.
- `File` tells FastAPI that the parameter must come from uploaded file data.
- `JSONResponse` returns controlled JSON output and status codes.

Service imports:

```
from services.database_service import DatabaseService
from services.decision_service import DecisionService
from services.file_service import FileService
from services.face_service import FaceVerificationService
from services.ocr_service import OCRService
```

Meaning:

- DatabaseService handles PostgreSQL search and verification.

- FileService validates and saves uploaded image files.
- FaceVerificationService compares uploaded document face with stored DB face.
- OCRService calls the external Indian ID validator OCR/model pipeline.

Object creation:

```
app = FastAPI()

database_service = DatabaseService()
decision_service = DecisionService()
file_service = FileService()
face_service = FaceVerificationService()
ocr_service = OCRService()
```

Meaning:

- `app` is the FastAPI server object.
- Service objects are created once and reused by API routes.
- This keeps the architecture modular.

Home endpoint:

```
@app.get("/")
def home():
    return {"message": "Identity Search API Running"}
```

This confirms the backend is running.

Manual search endpoint:

```
@app.post("/search-identity")
async def search_identity(field: str = Form(...), value: str = Form(...)):
```

This accepts `field` and `value` from frontend form data.

Inside it:

```
    results = database_service.search_identity(field, value)
```

This delegates all search logic to `DatabaseService`.

The endpoint returns:

```
{
    "status": "success",
    "total_matches": len(results),
    "results": results
}
```

Document validation endpoint:

```
@app.post("/validate-id")
async def validate_id(document_type: str = Form(...), document: UploadFile = File(...)):
```

Step 1:

```
saved_file_path = file_service.save_upload(document)
```

The uploaded image is validated and saved locally.

Step 2:

```
extracted_text = ocr_service.extract_text(saved_file_path, document_type)
```

The external Indian ID validator model is called. It extracts document fields and saves/exposes the detected face.

Step 3:

```
extracted_data = ocr_service.extract_identity_fields(document_type,  
extracted_text)
```

Raw model output is normalized into our internal format:

- full_name
- date_of_birth
- aadhar_number
- pan_number
- raw_text
- model_output

Step 4:

```
database_match = database_service.verify_identity_document(extracted_data)
```

The database is searched using Aadhaar or PAN depending on document type.

Step 5:

```
face_result = {  
    "matched": False,  
    "score": 0.0,  
    "method": "opencv_histogram_ncc",  
    "error": "Face verification skipped because database match was not found"  
}
```

This is a safe default. If no database match exists, face verification is skipped.

Step 6:

```
if database_match:  
    face_result = face_service.compare_faces(  
        ocr_service.get_last_face_path(),  
        database_match.get("photo_path")  
    )
```

If a database record exists, compare uploaded document face with the database photo.

The final result is generated.

Returned JSON:

```
{
  "status": "success",
  "extracted_data": extracted_data,
  "database_match": database_match,
  "face_verification": face_result,
  "decision": decision
}
```

Error handling:

- `ValueError` returns HTTP 400.
- Any unexpected exception returns HTTP 500.

This prevents the frontend from crashing silently.

9. File Upload Service: backend/services/file_service.py

Purpose:

The file service validates and saves uploaded document images.

Imports:

```
import os
import uuid
```

Meaning:

- `os` handles folders and file paths.
- `uuid` generates unique filenames so uploads do not overwrite each other.

Class:

```
class FileService:
```

This keeps upload logic separate from API logic.

Constructor:

```
def __init__(self, upload_dir="uploads"):
    self.upload_dir = upload_dir
    os.makedirs(self.upload_dir, exist_ok=True)
```

Meaning:

- Uploads are stored in `uploads`.
- The folder is created automatically if missing.

This receives FastAPI's `UploadFile`.

Extension extraction:

```
original_name = uploaded_file.filename or "uploaded_document"
extension = os.path.splitext(original_name)[1].lower()
```

This reads the file extension safely.

Allowed extensions:

```
allowed_extensions = {".jpg", ".jpeg", ".png"}
```

The system currently supports image uploads only.

Validation:

```
if extension not in allowed_extensions:
    raise ValueError("Only JPG, JPEG, and PNG uploads are supported")
```

This prevents unsupported files from entering the OCR pipeline.

Unique filename:

```
file_name = f"{uuid.uuid4().hex}{extension}"
```

This avoids filename conflicts.

Save file:

```
with open(file_path, "wb") as file_object:
    file_object.write(uploaded_file.file.read())
```

The uploaded binary file is written to disk.

Return:

```
return os.path.abspath(file_path)
```

The absolute path is returned so OCR can reliably find the file.

10. OCR Service: backend/services/ocr_service.py

Purpose:

OCRService connects this project to the existing `indian-id-validator` model pipeline. It avoids duplicating OCR logic and reuses the already trained YOLO and PaddleOCR setup.

Important imports:

```
import re
import json
import os
```


- `re` normalizes Aadhaar, PAN, and DOB patterns.
- `json` reads model output.
- `os` handles paths.
- `subprocess` runs the external `inference.py`.
- `sys` gives fallback Python executable.

Constructor:

```
self.validator_root = os.environ.get(
    "INDIAN_ID_VALIDATOR_ROOT",
    r"C:\AIProjects\indian-id-validator\model\indian-id-validator"
)
```

This points to the external model folder.

```
self.validator_python = os.environ.get(
    "INDIAN_ID_VALIDATOR_PYTHON",
    r"C:\AIProjects\indian-id-validator\venv\Scripts\python.exe"
)
```

This uses the external project's virtual environment because it already contains PaddleOCR, PaddlePaddle, Ultralytics, and model dependencies.

```
self.last_face_path = None
```

This stores the latest extracted face path after OCR runs.

Main OCR method:

```
def extract_text(self, image_path, document_type=None):
    return self._run_validator_model(image_path, document_type)
```

This calls the external model.

Field extraction:

```
def extract_identity_fields(self, document_type, extracted_text):
```

This converts model output into our standard format.

If output is already a dictionary:

```
if isinstance(extracted_text, dict):
    model_data = extracted_text
```

If not, it wraps the text:

```
model_data = {"raw_text": extracted_text}
```

Raw output for display/debug:

```
raw_text = json.dumps(model_data, indent=2)
```

Document type normalization:

Aadhaar extraction:

```
aadhaar_number = self._clean_aadhaar(  
    self._get_value(model_data, ["Aadhaar", "aadhaar", "Aadhaar Number", "UID"])  
)
```

The model may return different key names. This tries multiple options.

PAN extraction:

```
pan_number = self._clean_pan(  
    self._get_value(model_data, ["PAN", "pan", "PAN Number", "pan_number"])  
)
```

This supports both standard and current sample PAN values.

Final extracted structure:

```
extracted_data = {  
    "document_type": normalized_type,  
    "raw_text": raw_text,  
    "model_output": model_data,  
    "full_name": self._get_value(model_data, ["Name", "name"]),  
    "date_of_birth": self._extract_dob(model_data),  
    "aadhar_number": aadhaar_number,  
    "pan_number": pan_number  
}
```

This is what the database verification code consumes.

External model run:

```
command = [  
    python_executable,  
    "inference.py",  
    image_path,  
    "--model",  
    self._get_model_name(document_type),  
    "--output-json",  
    output_json  
]
```

This command runs the model with the selected document type.

Why subprocess:

The model pipeline belongs to a different project and virtual environment. Calling it through subprocess avoids dependency conflicts and keeps this service stable.

Run command:

```
result = subprocess.run(  
    command,  
    cwd=self.validator_root,
```

)

Meaning:

- `cwd` runs inside the model folder so model paths work.
- `capture\_output=True` captures model output/errors.
- `text=True` returns output as strings.

Error check:

```
if result.returncode != 0:  
    raise RuntimeError(...)
```

If OCR fails, API returns a controlled error.

Face path:

```
face_path = os.path.join(self.validator_root, "results", "face.jpg")
```

The existing model extracts the face and saves it here.

Before every run:

```
if os.path.exists(face_path):  
    os.remove(face_path)
```

This avoids using an old face from a previous request.

After run:

```
if os.path.exists(face_path):  
    self.last_face_path = face_path
```

This stores the newly extracted face.

Aadhaar cleaner:

```
digits = re.sub(r"\D", "", value)
```

This removes spaces and non-digits, allowing `9655 6981 6934` and `965569816934` to match.

PAN cleaner:

```
normalized_value = re.sub(r"[^A-Z0-9]", "", value.upper())  
match = re.search(r"[A-Z]{5}[0-9]{4}[A-Z]?", normalized_value)
```

This normalizes the PAN and supports both current sample values and standard PAN-like format.

DOB extraction:

The service checks direct DOB keys first, then scans all extracted values for date patterns.

Imports:

```
import psycopg2
import re
```

Meaning:

- `psycopg2` connects to PostgreSQL.
- `re` normalizes Aadhaar, PAN, and phone input.

Connection:

```
self.connection = psycopg2.connect(
    host="localhost",
    database="excel_import",
    user="postgres",
    password="postgres",
    port="5432"
)
```

This connects to the local PostgreSQL database.

Cursor:

```
self.cursor = self.connection.cursor()
```

The cursor executes SQL commands.

Allowed fields:

```
allowed_fields = {
    "full_name": "full_name",
    "date_of_birth": "date_of_birth",
    "dob": "date_of_birth",
    "aadhar_number": "aadhar_number",
    "aadhaar": "aadhar_number",
    "pan_number": "pan_number",
    "pan": "pan_number",
    "phone_number": "phone_number",
    "phone": "phone_number",
    "email": "email",
    "employee_id": "employee_id",
    "department": "department",
    "state": "state"
}
```

Why this exists:

It prevents SQL injection through column names. Only approved field names can be searched.

Unknown field:

```
if field not in allowed_fields:
    return []
```

Aadhaar and phone normalization:

```
where_clause = f"REGEXP_REPLACE(CAST({db_column} AS TEXT), '\\D', '', 'g') ILIKE %s"
search_value = re.sub(r"\D", "", search_value)
```

This removes spaces and symbols from the database value and user/OCR value before comparison.

Why this matters:

Database may store Aadhaar as `9655 6981 6934`, while OCR extracts `965569816934`. Normalization makes both comparable.

PAN normalization:

```
where_clause = f"UPPER(REGEXP_REPLACE(CAST({db_column} AS TEXT), '\\s', '', 'g')) ILIKE %s"
search_value = re.sub(r"\s+", "", search_value).upper()
```

This ignores spaces and case in PAN matching.

SQL query:

```
SELECT
    employee_id,
    full_name,
    date_of_birth,
    aadhar_number,
    pan_number,
    phone_number,
    email,
    department,
    state,
    photo_path
FROM demodataset
WHERE {where_clause}
LIMIT 20
```

This returns identity details and photo path.

Parameterized execution:

```
self.cursor.execute(query, (f"%{search_value}%",))
```

The value is passed separately from SQL to reduce injection risk.

Result formatting:

Each row is converted into a dictionary so FastAPI can return JSON and Streamlit can display it.

Document verification:

```
def verify_identity_document(self, extracted_data):
```

For Aadhaar:

```
search_field = "aadhaar"  
search_value = extracted_data.get("aadhar_number")
```

For PAN:

```
search_field = "pan"  
search_value = extracted_data.get("pan_number")
```

If no value:

```
return None
```

If matches exist:

```
return matches[0]
```

The first matching database record becomes the verified candidate.

12. Face Service: backend/services/face_service.py

Purpose:

This service compares the face extracted from the uploaded document with the stored database face image.

Imports:

```
import os  
import cv2  
import numpy as np
```

Meaning:

- `os` resolves file paths.
- `cv2` loads images, detects faces, resizes, and calculates histograms.
- `numpy` calculates scores.

Constructor:

```
def __init__(self, match_threshold=0.60, target_size=(128, 128)):
```

Meaning:

- Score above 0.60 is considered a match.
- Both face images are resized to 128x128 for fair comparison.

Face detector:

```
self.face_cascade = cv2.CascadeClassifier(  
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"  
)
```

```
def compare_faces(self, uploaded_face_path, database_face_path):
```

This receives:

- Face extracted from uploaded document.
- Photo path from database record.

Path resolving:

```
resolved_database_path = self.resolve_database_photo_path(database_face_path)
```

The database may store relative paths like:

```
/matched_employee_photos/IA202610012/IA202610012_face.jpg
```

The service resolves that to the local frontend folder.

Image loading:

```
uploaded_image, uploaded_error = self._load_image(uploaded_face_path)
database_image, database_error = self._load_image(resolved_database_path)
```

If either image cannot load, the method returns a safe error result.

Face extraction:

```
uploaded_face = self._extract_face(uploaded_image)
database_face = self._extract_face(database_image)
```

If no face is detected, it falls back to the full image. This prevents crashes.

Resize:

```
uploaded_face = cv2.resize(uploaded_face, self.target_size)
database_face = cv2.resize(database_face, self.target_size)
```

Both images are standardized.

Blended score:

```
score = self._blended_score(uploaded_face, database_face)
```

The score combines:

- 40 percent color histogram similarity.
- 60 percent structural grayscale similarity.

Histogram score:

```
cv2.calcHist(...)
cv2.compareHist(..., cv2.HISTCMP_CORREL)
```

This compares color distribution.

Final output:

```
{
  "matched": score >= self.match_threshold,
  "score": round(score, 4),
  "threshold": self.match_threshold,
  "method": "opencv_histogram_ncc",
  "uploaded_face_path": uploaded_face_path,
  "database_face_path": resolved_database_path,
  "error": None
}
```

This is used by the decision service and UI.

Important client explanation:

This is a lightweight prototype face-matching method. It is designed to demonstrate the verification workflow without installing heavy face recognition frameworks. In production, the internal implementation can be upgraded to InsightFace, DeepFace, FaceNet, or another embedding-based face recognition model without changing the API or frontend flow.

13. Decision Service: backend/services/decision_service.py

Purpose:

This service centralizes the final verification logic.

Main method:

```
def build_decision(self, database_match, face_result=None):
```

If no database match:

```
return {
    "status": "NOT VERIFIED",
    "message": "No matching user found in database"
}
```

If database match and face matched:

```
return {
    "status": "VERIFIED",
    "message": "User exists in database and face matched"
}
```

If database match but face not confident:

```
return {
    "status": "MANUAL REVIEW",
    "message": "User exists in database, but face did not match confidently"
}
```

Why this is important:

14. Frontend: frontend/dashboard.py

Purpose:

This file builds the Streamlit user interface.

Imports:

```
import streamlit as st
import requests
import os
```

Meaning:

- `streamlit` builds the dashboard.
- `requests` calls the backend API.
- `os` checks image paths before displaying images.

API URLs:

```
API_URL = "http://127.0.0.1:8000/search-identity"
VALIDATE_ID_URL = "http://127.0.0.1:8000/validate-id"
```

These point frontend actions to backend endpoints.

Page setup:

```
st.set_page_config(
    page_title="Identity Search System",
    layout="wide"
)
```

This sets browser title and wide layout.

Tabs:

```
search_tab, document_tab = st.tabs(
    ["Search Identity", "Validate Uploaded Document"]
)
```

The UI has two workflows:

- Manual search.
- Document verification.

Search options:

```
search_options = {
    "Full Name": "full_name",
    "DOB": "date_of_birth",
    "Aadhaar Number": "aadhaar_number",
    "PAN Number": "pan_number",
}
```

This maps user-friendly labels to backend fields.

Display function:

```
def display_person(person):
```

This displays a database record with image and identity details.

Image handling:

```
corrected_path = os.path.abspath(photo_path.rstrip("/"))
```

This tries to convert database paths into local filesystem paths.

Manual search flow:

- User selects search type.
- User enters search value.
- Streamlit posts to `/search-identity`.
- Results are displayed using `display\_person`.

Document upload flow:

```
uploaded_document = st.file_uploader(  
    "Upload Aadhaar or PAN image",  
    type=["jpg", "jpeg", "png"]  
)
```

This accepts image uploads.

Validation button:

```
if st.button("Validate Document"):
```

When clicked, the UI sends the file to FastAPI.

API call:

```
response = requests.post(  
    VALIDATE_ID_URL,  
    data={"document_type": document_type},  
    files={"document": (...)}  
)
```

This sends both selected document type and image file.

Result display:

- Shows final decision message.
- Shows OCR Raw Text in an expander.
- Shows document face and database face side by side.
- Shows whether the face matched.
- Shows database user details when database match exists.

15. Database Design

Database name:

`excel_import`

Main table:

`demodataset`

Important columns:

- `employee_id`
- `full_name`
- `date_of_birth`
- `aadhar_number`
- `pan_number`
- `phone_number`
- `email`
- `department`
- `state`
- `photo_path`

The ``photo_path`` column links the identity record to a stored face image.

Example path:

`/matched_employee_photos/IA202610012/IA202610012_face.jpg`

16. Verification Workflow In Detail

Step 1: User uploads Aadhaar or PAN image.

Step 2: FastAPI validates the upload extension.

Step 3: File is saved with a UUID filename.

Step 4: OCRService calls:

`C:\AIPProjects\indian-id-validator\model\indian-id-validator\inference.py`

Step 5: The external model detects document fields and runs OCR.

Step 6: The model output is parsed into our standard identity dictionary.

Step 7: DatabaseService checks Aadhaar or PAN against PostgreSQL.

Step 8: If database match exists, FaceVerificationService compares faces.

Step 9: DecisionService returns final decision.

Step 10: Streamlit displays result

Problem:

OCR may return:

```
965569816934
```

Database may store:

```
9655 6981 6934
```

Solution:

Both are normalized by removing non-digits.

In OCR:

```
re.sub(r"\D", "", value)
```

In SQL:

```
REGEXP_REPLACE(CAST(aadhar_number AS TEXT), '\D', '', 'g')
```

Result:

Both become:

```
965569816934
```

Then matching succeeds.

18. Handling PAN Format Differences

Problem:

PAN can appear in OCR with spaces, mixed case, or a non-standard sample pattern.

Solution:

The system:

- Converts PAN to uppercase.
- Removes spaces and symbols.
- Accepts current sample pattern.
- Searches database with normalized uppercase PAN.

19. Face Verification Approach

The existing ID validator extracts a face from the uploaded document and saves:

Identity Search And Document Verification System | Page 20
C:\AIPProjects\indian-id-validator\model\indian-id-validator\results\face.jpg

- Detects the largest face if possible.
- Crops the face region.
- Resizes both images to 128x128.
- Calculates color histogram similarity.
- Calculates structural grayscale similarity.
- Combines both into one score.
- Compares against threshold 0.60.

Why this approach:

- It avoids heavy dependencies.
- It is fast for a prototype.
- It works with local images.
- It does not break the existing architecture.
- It can be upgraded later.

20. Final Decision Logic

NOT VERIFIED:

Database match is not found.

MANUAL REVIEW:

Database match is found, but face confidence is low.

VERIFIED:

Database match is found and face comparison is successful.

This is a practical workflow because document text match and face match are different evidence layers.

21. How To Run

Install packages:

```
cd c:\AIPProjects\identity-search-service
pip install streamlit fastapi uvicorn pycpg2-binary python-multipart requests
opencv-python numpy
```

Start backend:

```
cd c:\AIPProjects\identity-search-service\backend
uvicorn app:app --reload
```

Start frontend:

```
cd c:\AIPProjects\identity-search-service\frontend
streamlit run dashboard.py
```

Open the Streamlit URL shown in the terminal

22. Demo Script You Can Speak To Client

Start:

"This is our Identity Search and Document Verification System. It is designed to verify whether a person from an uploaded Aadhaar or PAN document exists in our internal employee database and whether their face matches the database photo."

Manual search:

"First, we can manually search existing employees by fields like name, Aadhaar, PAN, phone, email, department, or employee ID. This helps an operator quickly inspect records."

Document upload:

"Now we move to document verification. I select the document type, upload the ID image, and the system sends it to the backend."

OCR:

"The backend saves the file and calls our existing Indian ID validator model. This model uses YOLO to detect fields on the document and PaddleOCR to read text from those fields."

Database verification:

"After OCR, we normalize values such as Aadhaar and PAN. For example, Aadhaar may be extracted without spaces, while the database stores it with spaces. Our system removes formatting differences before matching."

Face verification:

"Once the database record is found, the system compares the face extracted from the uploaded document with the stored employee photo. It shows both images side by side so the operator can visually confirm the evidence."

Decision:

"The final decision is rule-based. If no database record exists, the result is NOT VERIFIED. If the database record exists but face confidence is low, the result is MANUAL REVIEW. If both database and face match, the result is VERIFIED."

Close:

"The important part is that the architecture is modular. OCR, database verification, face verification, file handling, and decision logic are separate services. This means we can later upgrade one component, such as replacing lightweight OpenCV face matching with a production-grade face embedding model, without rewriting the whole application."

23. Strengths To Highlight

- Modular service-based architecture.
- Uses existing trained OCR and document detection model.

- Face verification adds second-level identity proof.
- Streamlit UI is simple and operator-friendly.
- FastAPI backend can be integrated with other systems later.
- Error handling prevents crashes.
- Future-ready design.

24. Limitations And Honest Positioning

Current limitations:

- PDF document upload is not enabled in this project flow.
- Face matching is prototype-level OpenCV comparison.
- Database credentials are hardcoded and should move to environment variables in production.
- Single shared PostgreSQL cursor is acceptable for prototype but should become per-request or pooled in production.
- Audit logging is not yet implemented.
- Role-based login is not yet implemented.

How to explain this:

"For prototype delivery, we focused on a working end-to-end verification workflow. The architecture is intentionally modular, so production hardening can be added without changing the user-facing workflow."

25. Future Enhancements

- PDF support.
- Login and role-based access.
- Audit trail for every verification.
- Human review queue for manual review cases.
- Stronger face recognition using embeddings.
- Docker deployment.
- Environment-based configuration.
- Database connection pooling.
- Report generation.
- Liveness detection.
- QR validation for Aadhaar.
- Fraud signal detection.

26. One-Line Summary

This system turns manual identity checking into an automated, evidence-based workflow by combining document OCR, database verification, face comparison, and a clean verification decision in one dashboard.